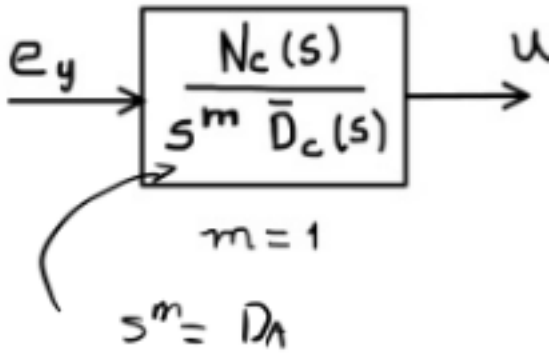
ADRC_ASM

July 30, 2026

1 CONTROL CON RECHAZO ACTIVO DE PERTURBACIONES PARA LA REGULACIÓN DE OXÍGENO DISUELTO EN LA ETAPA DE LODOS ACTIVADOS DE UNA PTAR





1.0.1 Mass balance

$$\text{Acumulación} = \text{Entrada} - \text{Salida} + \text{Generación} - \text{Consumo}$$

$$\frac{dS}{dt} = \frac{1}{V}\sum_{i=1}^{3} Q_{in,i}S_{in,i} - \frac{Q_{out}}{V}S + K_{L_a}\left(S^* - S\right) - r_z(S)$$

$$\underbrace{\frac{dS}{dt}}_{\frac{\text{mg}}{\text{L·h}}} = \underbrace{\frac{1}{V}\sum_{i=1}^{3} Q_{in,i}S_{in,i}}_{\frac{1}{\text{L}}\left(\frac{\text{L}}{\text{h}}\right)\left(\frac{\text{mg}}{\text{L}}\right)=\frac{\text{mg}}{\text{L·h}}} - \underbrace{\frac{Q_{out}}{V}S}_{\left(\frac{\text{L/h}}{\text{L}}\right)\left(\frac{\text{mg}}{\text{L}}\right)=\frac{\text{mg}}{\text{L·h}}} + \underbrace{K_{La}(S^* - S)}_{\left(\frac{1}{\text{h}}\right)\left(\frac{\text{mg}}{\text{L}}\right)=\frac{\text{mg}}{\text{L·h}}} - \underbrace{r_z(S)}_{\frac{\text{mg}}{\text{L·h}}}$$

With r_Z :

$$r_Z = -\frac{1 - Y_H}{Y_H}\,\rho_1 - \frac{4.57 - Y_A}{Y_A}\,\rho_3 \tag{1}$$

$$\rho_1 = \mu_H \left(\frac{S_S}{K_S + S_S}\right)\left(\frac{S_O}{K_{O,H} + S_O}\right) X_{B,H} \tag{2}$$

$$\rho_3 = \mu_A \left(\frac{S_{NH}}{K_{NH} + S_{NH}}\right)\left(\frac{S_O}{K_{O,A} + S_O}\right) X_{B,A} \tag{3}$$

If the oxygen transfer coefficient is considered proportional to the control input,

$$K_{La} \propto u \tag{4}$$

then,

$$K_{La} = \alpha u \tag{5}$$

where α is a proportionality constant.

Substituting into the dissolved oxygen mass balance yields

$$\dot{S} = \underbrace{\frac{1}{V}\sum_{i=1}^{3} Q_{in,i}S_{in,i} - \frac{Q_{out}}{V}S - r_z}_{f(S,t)} + u\,\underbrace{\alpha\left(S^* - S\right)}_{g(S,t)}. \tag{6}$$

$$\dot{S} = f(S,t) + g(S,t)\,u \tag{7}$$

where

$$g(S,t) = \alpha\left(S^* - S\right) \tag{8}$$

and

$$f(S,t) = \frac{1}{V}\sum_{i=1}^{3} Q_{in,i}S_{in,i} - \frac{Q_{out}}{V}S - r_z(S) \tag{9}$$

All dynamics that differ from a pure integrator are grouped as a total disturbance f,

$$\dot{y} = f + b_0 \cdot K_{L_a} \tag{10}$$

where the total disturbance f is defined as:

$$f(S,t) = \frac{1}{V}\sum_{i=1}^{3} Q_{in,i} S_{in,i} - \frac{Q_{out}}{V} S - r_z(S) + K_{L_a}(S^* - S) - b_0 \cdot K_{L_a} \tag{11}$$

relationship between Oxygen transfer coefficient K_{La} and u. Oxygen transfer coefficient is proportional to the control signal u applied to the blowers:

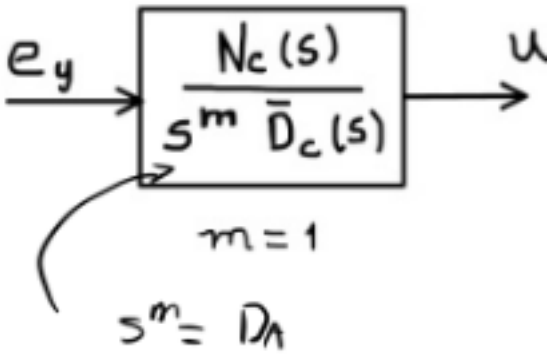
$$K_{L_a} \propto u \tag{12}$$

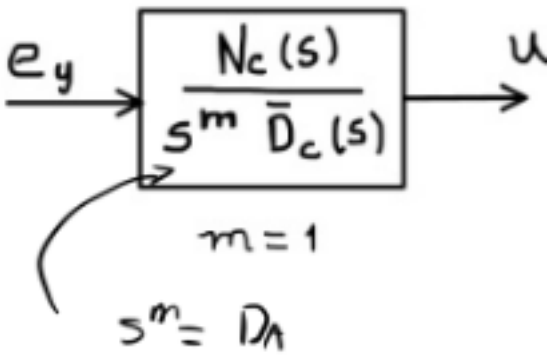
$$K_{L_a} = \alpha \cdot u \tag{13}$$

with $k = bo \cdot \alpha$

$$\dot{y} = f + k \cdot u$$

1.0.2 Control Law with observer





Now, we have to design to the ADRC We rewrite the second-order system as a second-order error system. Let $e_y = y - r$:

$$\dot{e}_y = k \cdot u + (\bar{\xi} - u^*)$$

Defining $\xi = \bar{\xi} - u^*$:

$$\dot{e}_y = k \cdot u + \xi$$

Applying the same procedure as before, the estimation error dynamics become: We have to select to the states of the system.

$$\dot{x_1} = x_2 + k \cdot u$$
$$\dot{x_2} = PMI(\xi, t)$$

Extended Dynamics System

$$\dot{X}_{au} = A_{au} \cdot X_{au} + B_{au} \cdot u + B_{\xi_{au}} \cdot \xi^{(m)}$$

with the state vectors defined as:

$$X_p = \left[e_y\right], \qquad X_\xi = \left[\xi\right]$$

and the augmented state:

$$X_{au} = \begin{bmatrix} X_p \\ X_\xi \end{bmatrix}$$

Observer System

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} + B_{au}u + Le_y$$

with $e_y = C_{au} \cdot X_{au}$

Estimation Error Dynamics

$$(\dot{X}_{au} - \dot{\widehat{X}}_{au}) = (A_{au} - LC_{au})(X_{au} - \widehat{X}_{au}) + B_{\xi_{au}}\xi^{(m)}$$

$$\dot{\bar{e}} = (A_{au} - LC_{au})\bar{e} + B_{\xi_{au}}\xi^{(m)}$$

The control law is given by the following expression, where the observer provides the estimated plant and disturbance states:

$$e^{(1)} = v \tag{14}$$

Ideal model:

$$f_{\delta}^{(n)} = \beta \cdot u + \phi(f, \dot{f}_{\delta}, ..., f_{\delta}^{(n-1)}) \tag{15}$$

Donde β, $f_{\delta}^{(n)}$ y ϕ corresponde a la dinámica no deseada. Ahora bien de aquí se puede establecer la señal de control.

$$u = \frac{1}{\beta}[-\phi - v(ef^{(n-1)}, .., ef, f^{(n)*})] \tag{16}$$

Now:

$$u = \frac{1}{k}[-v(ey^{(0)}, \xi)] \tag{17}$$

$$u = \frac{1}{k}[-v(\widehat{X}_{au})] \tag{18}$$

with v:

$$v = k_1 \cdot X_{au_1} + k_2 \cdot \widehat{X}_{au_2}$$

$$v = K \cdot \widehat{X}_{au}$$

1.0.3 Discrte time



Disturbance Estimator

$$x[k+1] = A\,x[k] + b(u[k] + \xi[k]) \tag{19}$$

Disturbance system:

$$x_{im}[k+1] = A_{im}\,x_{im}[k] \tag{20}$$
$$\xi[k] = C_{im}\,x_{im} \tag{21}$$

Augmented system:

$$z[k+1] = A\,x[k] + b\,C_{im}\,x_{im}[k] + b\,u[k] \tag{22}$$

In matrix form:

$$\begin{bmatrix} x[k+1] \\ x_{im}[k+1] \end{bmatrix} = \underbrace{\begin{bmatrix} A & b\,C_{im} \\ 0 & A_{im} \end{bmatrix}}_{A_{ou}} \underbrace{\begin{bmatrix} x[k] \\ x_{im}[k] \end{bmatrix}}_{x_{au}} + \underbrace{\begin{bmatrix} b \\ 0 \end{bmatrix}}_{b_{au}} u \tag{23}$$

$$y = \underbrace{\begin{bmatrix} C & 0 \end{bmatrix}}_{C_{au}} \begin{bmatrix} x[k] \\ x_{im}[k] \end{bmatrix} \tag{24}$$

with Observer

Augmented system:

$$x_{au}[k+1] = A_{au}\,x_u + b_{au}\,u[k] \tag{25}$$

Observer equation:

$$\hat{x}_{au}[k+1] = A_{au}\,\hat{x}_{au}[k] + b_{au}\,u[k] + L_{au}(e[k] - \hat{e}[k]) \tag{26}$$

Expanded form:

$$\hat{x}_{au}[k+1] = (A_{au} - L_{au}\,C_{au})\,\hat{x}_{au} + b_{au}\,u[k] - L_{au}\,e[k] \tag{27}$$

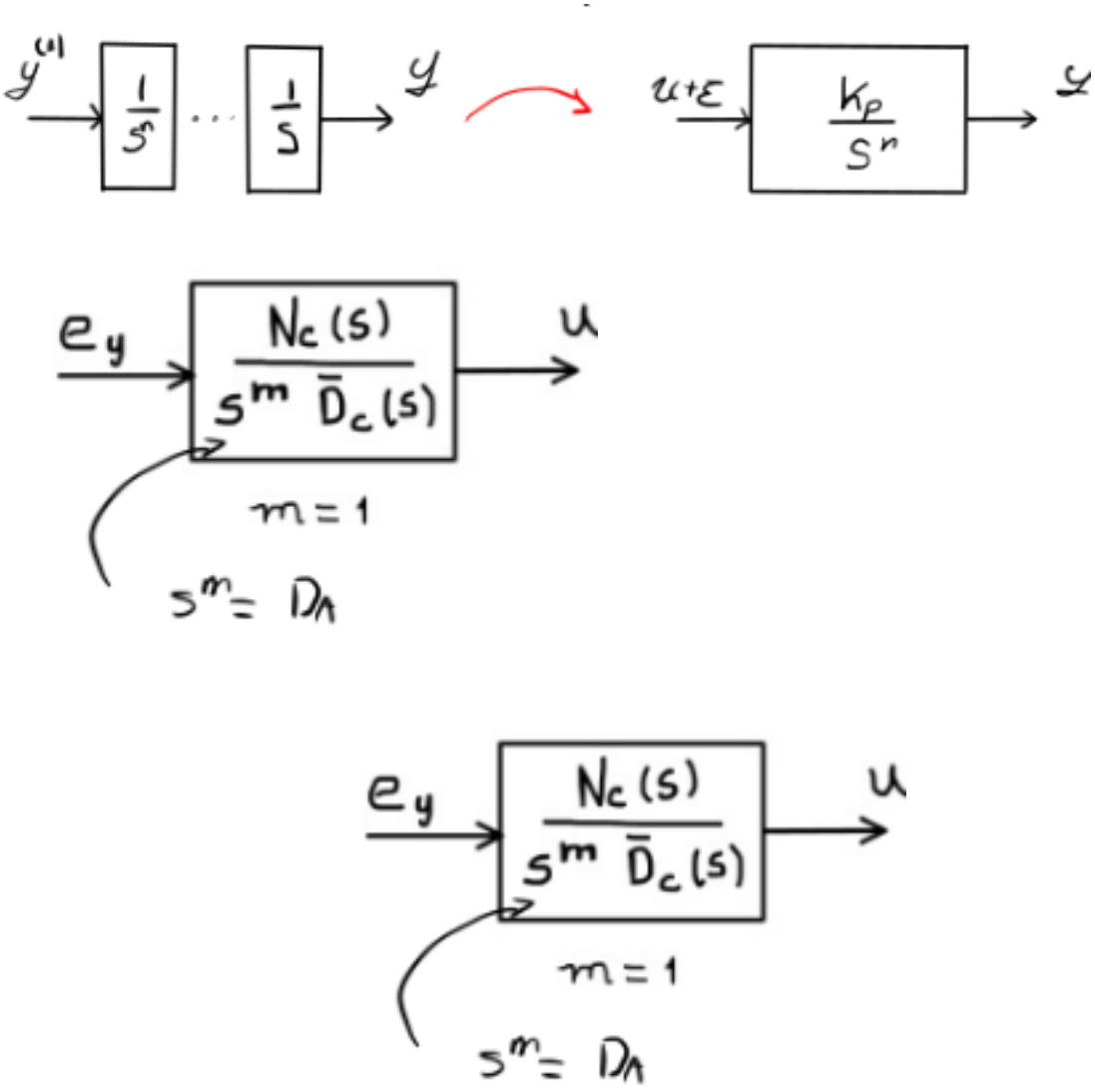
Control Law

$$u = \frac{1}{k_{sys}}[-v(ey^{(0)}[k], \xi[k])] \tag{28}$$

$$u = \frac{1}{\kappa_{sys}}\left[-k_1\,\hat{x}_{ou_1}[k] + k_2\,\hat{x}_{ou}[k]\right] \tag{29}$$

$$u = \frac{1}{\kappa_{sys}}\left[-k_1\,\hat{e}_y[k] + k_2\,\hat{\xi}[k]\right] \tag{30}$$

1.0.4 Control Law with GPI



1.0.5 COntrol law

$$u(s) = -\frac{1}{\kappa}\left[\frac{k_{n+m-1}s^{n+m-1} + ... + k_1 s + k_0}{s^m\left(s^{n-1} + k_{2n+m-2}s^{n-2} + ... + k_{n+m}\right)}\right] e_y(s) \tag{31}$$

$$u(s) = -\frac{1}{\kappa}\left[\frac{k_1 s + k_0}{s\,(k_2)}\right] e_y(s) \tag{32}$$

1.0.6 Discrete Time

Applying the Tustin discretization, the continuous operator s is replaced by:

$$s \longleftarrow \frac{2}{T_s}\cdot\frac{z-1}{z+1} \tag{33}$$

where T_s is the sampling period. Substituting into the controller:

$$u(z) = -\frac{1}{\kappa}\left[\frac{k_1\left(\frac{2}{T_s}\cdot\frac{z-1}{z+1}\right)+k_0}{k_2\left(\frac{2}{T_s}\cdot\frac{z-1}{z+1}\right)}\right]e_y(z) \tag{34}$$

Simplifying:

$$u(z) = -\frac{1}{\kappa\, k_2}\left[\frac{\left(\frac{2k_1}{T_s}+k_0\right)z+\left(k_0-\frac{2k_1}{T_s}\right)}{\frac{2}{T_s}(z-1)}\right]e_y(z) \tag{35}$$

In difference equation form :

$$u[k] = u[k-1] - \frac{T_s}{2\,\kappa\, k_2}\left[\left(\frac{2k_1}{T_s}+k_0\right)e_y[k] + \left(k_0-\frac{2k_1}{T_s}\right)e_y[k-1]\right] \tag{36}$$

1.0.7 Simulacion del Modelo BSM1

1.0.8 ADRC digital basado en el error con observador (orden 2)

Siguiendo la estructura del **ADRC digital del motor** (control basado en el error con observador), aqui se implementa una version reducida para el oxigeno disuelto:

- **Planta de orden 1**: $\dot{y} = f + b_0\, u$, con $y = S_O$ y $b_0 = \alpha\,(S^* - r)$.
- **Perturbacion muy lenta de orden 1**: $\dot{f} \approx 0$ (modelo interno integrador).
- **Observador de estado extendido (ESO) de orden 2**: estima $[\,y,\ f\,]$.

Estado aumentado $x = [\,y;\ f\,]$:

$$A_{au} = \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}, \qquad B_{au} = \begin{bmatrix}b_0\\ 0\end{bmatrix}, \qquad C_{au} = \begin{bmatrix}1 & 0\end{bmatrix}.$$

Las ganancias del observador L se obtienen por **ubicacion de polos (`acker`)** sobre el sistema aumentado discretizado (mismo procedimiento del MATLAB). La ley de control cancela la perturbacion estimada y cierra un lazo de primer orden:

$$u = \frac{1}{b_0}[\,\omega_c\,(r-\hat{y}) - \hat{f}\,].$$

[1]:
```
import numpy as np
from sympy import *
from sympy import Matrix, symbols, init_printing
from sympy import Function, laplace_correspondence, diff, solve,symbols
import matplotlib.pyplot as plt
from control.matlab import *
from IPython.display import display, Latex
```

```
init_printing(use_latex='mathjax')
```

[2]:
```
# ============================================================
# Other functions
# ============================================================

def tf2sym(tf,symb=None):
    s = symbols(symb) if symb is not None else symbols('s')
    numTF = tf.num[0][0]
    denTF = tf.den[0][0]
    num_symb = sum(numTF[i]*s**(len(numTF)-1-i)
                   for i in range(len(numTF)))
    den_symb = sum(denTF[i]*s**(len(denTF)-1-i)
                   for i in range(len(denTF)))
    return (num_symb / den_symb).simplify()

def sym2tf(expr,symb=None,Ts=None):

    s = symbols(symb) if symb is not None else symbols('s')
    num, den = fraction(expr)
    num_poly = Poly(num, s)
    den_poly = Poly(den, s)

    num_coeff = [float(i) for i  in num_poly.as_poly().all_coeffs()]
    den_coeff = [float(i) for i in den_poly.as_poly().all_coeffs()]
    if Ts!=None:
        return tf(num_coeff,den_coeff,Ts)
    else:
        return tf(num_coeff,den_coeff)
```

[3]:
```
"""
Nonlinear DO mass-balance - single CSTR reactor (compartment 5 of BSM1).
Based on the user's equation from the notebook:

    dS/dt = (1/V) * sum_{i=1}^{3} Q_{in,i} * S_{in,i}
            - (Q_out/V) * S
            + KLa * (S* - S)
            - rZ(S)

UNITS
-----
    Volume   : L          (1 m³ = 1000 L)
    Flow     : L/h
    Conc.    : mg/L  (= g/m³, no conversion needed)
    Time     : h
    Rates    : mg/(L·h)
    KLa      : 1/h
```

```
CONTROL SIGNAL
--------------
    KLa [1/h] = alpha * u,    alpha = 1/30

THREE INLETS
------------
    Q1, S_O1 : influent  (fresh feed - low DO)
    Q2, S_O2 : return sludge from clarifier  (low DO)
    Q3, S_O3 : internal recycle from upstream aerated zone
    Q_out = Q1 + Q2 + Q3  (mass balance)

DISTURBANCE
-----------
    Driven by BSM1 dry-weather influent data:
      - Q1(t) : influent flow varies diurnally
      - Ss(t) : influent substrate varies diurnally
    (Q2, Q3 kept at nominal BSM1 ratios)
"""

import numpy as np

# ---------------------------------------------------------------
# HYDRAULICS  - nominal BSM1 dry-weather values
#   original in m³/d → converted to L/h: × (1000/24)
# ---------------------------------------------------------------
_Q1_nom = 18446.0 * 1000.0 / 24.0   # L/h  influent (base)
_Q2     = 18446.0 * 1000.0 / 24.0   # L/h  return sludge (constant)
_Q3     = 55338.0 * 1000.0 / 24.0   # L/h  internal recycle (constant)

V = 1333.0 * 1000.0   # L   (single aerated reactor, 1333 m³)

# DO concentrations of the three inlet streams [mg/L]
S_O1 = 0.0    # influent - essentially no dissolved oxygen
S_O2 = 0.0    # return sludge - no oxygen at clarifier underflow
S_O3 = 1.5    # internal recycle - partial DO from upstream zone

S_O_SAT = 8.0   # mg/L  oxygen saturation at 15 °C

# Control-signal / KLa relationship
ALPHA = 1.0 / 30.0   # KLa [1/h] = ALPHA * u

# ---------------------------------------------------------------
# ASM1 KINETICS  - BSM1 default at 15 °C
#   rates originally 1/d → divided by 24 to get 1/h
# ---------------------------------------------------------------
mu_H = 4.0  / 24.0   # 1/h  heterotrophic max growth rate
```

```
mu_A = 0.5  / 24.0   # 1/h  autotrophic max growth rate
K_S  = 10.0           # mg/L  half-saturation substrate
K_OH = 0.2            # mg/L  half-saturation O2 (hetero)
K_NH = 1.0            # mg/L  half-saturation NH4 (auto)
K_OA = 0.4            # mg/L  half-saturation O2 (auto)
Y_H  = 0.67           # g cell COD / g COD removed
Y_A  = 0.24           # g cell COD / g N oxidised

# ---------------------------------------------------------------
# QUASI-STEADY BIOMASS inside the reactor  (BSM1 default)
# ---------------------------------------------------------------
_Ss_nom = 1.0      # mg/L  readily biodegradable substrate in reactor
S_NH    = 1.0      # mg/L  ammonia nitrogen in reactor
X_BH    = 2500.0   # mg/L  active heterotrophic biomass
X_BA    = 150.0    # mg/L  active autotrophic biomass


# ---------------------------------------------------------------
# ASM1 reaction rates
# ---------------------------------------------------------------
def rho1(S_O, Ss):
    """Heterotrophic O2 uptake rate [mg/(L·h)]."""
    return mu_H * (Ss / (K_S + Ss)) * (S_O / (K_OH + S_O)) * X_BH


def rho3(S_O):
    """Autotrophic O2 uptake rate [mg/(L·h)]."""
    return mu_A * (S_NH / (K_NH + S_NH)) * (S_O / (K_OA + S_O)) * X_BA


def rZ(S_O, Ss):
    """Net biological O2 consumption [mg/(L·h)]  (negative = consumption)."""
    return -(1 - Y_H) / Y_H * rho1(S_O, Ss) - (4.57 - Y_A) / Y_A * rho3(S_O)


# ---------------------------------------------------------------
# BSM1 dry-weather disturbance loader
# ---------------------------------------------------------------
def load_bsm1_disturbance(filepath):
    """
    Load BSM1 influent data.
    Columns: t[d]  Si  Ss  Xi  Xs  Xbh  Xba  Xp  So  Sno  Snh  Snd  Xnd  Salk␣
 ↪Q[m3/d]
    Returns arrays with t in HOURS and Q in L/h.
    """
    raw = np.loadtxt(filepath, skiprows=1)
    t_h  = raw[:, 0]  * 24.0            # days → hours
```

```
    Ss_t = raw[:, 2]                    # mg/L  (g/m³ = mg/L, no change)
    Q_t  = raw[:, 14] * 1000.0 / 24.0  # m³/d → L/h
    return t_h, Ss_t, Q_t


def make_disturbance_interp(filepath):

    t_h, Ss_t, Q_t = load_bsm1_disturbance(filepath)

    # tiempo en HORAS (consistente con la simulacion; antes se mezclaba con␣
 ↪segundos)
    t_max = t_h[-1]

    def Ss_func(t):
        return float(np.interp(t % t_max, t_h, Ss_t))

    def Q1_func(t):
        return float(np.interp(t % t_max, t_h, Q_t))

    return Ss_func, Q1_func


# ------------------------------------------------------------------
# Main ODE  (call each discrete time step)
# ------------------------------------------------------------------
def dSO5_dt(t, S_O, u, Ss_func=None, Q1_func=None):
    """
    Time derivative of dissolved oxygen in the CSTR [mg/(L·h)].

    Parameters
    ----------
    t        : current time [h]
    S_O      : current DO concentration [mg/L]
    u        : control signal (dimensionless);  KLa = ALPHA * u  [1/h]
    Ss_func  : callable(t) → influent Ss [mg/L]  (None → nominal 63.6)
    Q1_func  : callable(t) → Q1 in L/h            (None → nominal)
    """
    # Flow
    Q1  = Q1_func(t) if Q1_func is not None else _Q1_nom
    Qt  = Q1 + _Q2 + _Q3           # total outflow = sum of inflows

    # Inlet DO mix
    Q_in_sum = Q1*S_O1 + _Q2*S_O2 + _Q3*S_O3

    # Reactor Ss:  influent Ss drives biological load.
    # A simple approximation: reactor Ss scales with influent Ss
    # (ratio reflects partial degradation through anoxic zone upstream).
```

```
    # Nominal influent Ss = 63.6 mg/L → reactor Ss  1.0 mg/L (factor  0.016)
    SS_FACTOR = 0.016
    Ss_inf   = Ss_func(t) if Ss_func is not None else 63.6
    Ss_react = max(Ss_inf * SS_FACTOR, 0.05)   # reactor substrate [mg/L]

    # Aeration
    KLa_h = max(ALPHA * u, 0.0)     # physical lower bound

    # DO mass balance (user's equation, 3 inlets)
    # Note: rZ() already returns a NEGATIVE value (signed O2 production rate,
    # negative because biology CONSUMES oxygen).  We add it directly.
    dS = ((1.0 / V) * Q_in_sum
          - (Qt / V) * S_O
          + KLa_h * (S_O_SAT - S_O)
          + rZ(S_O, Ss_react))     # ← + rZ  (rZ < 0, so this subtracts O2)

    return dS


def steady_state_u(S_O_target=2.0):
    """Steady-state blower signal at nominal Q1."""
    Qt  = _Q1_nom + _Q2 + _Q3
    Q_in_sum = _Q1_nom*S_O1 + _Q2*S_O2 + _Q3*S_O3
    # 0 = (1/V)*Q_in - (Qt/V)*S + KLa*(S*-S) + rZ(S)  ← rZ < 0
    lhs = (Qt / V) * S_O_target - (1.0/V)*Q_in_sum - rZ(S_O_target, _Ss_nom)
    KLa_ss = lhs / (S_O_SAT - S_O_target)
    return KLa_ss / ALPHA
```

[4]:
```
# ============================================================
# ADRC digital basado en el error con observador (ESO)
#   Planta orden 1  +  perturbacion lenta orden 1  ->  observador orden 2
#   Ubicacion de polos del observador con acker (igual que el MATLAB).
#   Modelo ADRC:  y_dot = f + b0*u ,   f_dot ~ 0   (perturbacion lenta)
#   Estado aumentado:  x = [ y ; f ]
# ============================================================
BSM1_FILE = 't_Si_Ss_Xi_Xs_Xbh_Xba.txt'
plt.rcParams.update({"figure.dpi":130,"font.size":9,"axes.grid":True,"grid.
 ↪alpha":0.35})

# ---- parametros de diseno ----------------------------------------
T_s   = 0.02                        # h   periodo de muestreo
r_nom = 2.0                         # mg/L set-point nominal
b0    = ALPHA*(S_O_SAT - r_nom)     # ganancia de entrada ~0.2  (de KLa*(S*-y))

# ---- sistema aumentado continuo  x=[y;f]:  y_dot=f+b0*u ,  f_dot=0
Ac = np.array([[0.0, 1.0],[0.0, 0.0]])
Bc = np.array([[b0],[0.0]])
```

```
Cc = np.array([[1.0, 0.0]])
sys_d = c2d(ss(Ac, Bc, Cc, 0.0), T_s, 'zoh')      # discretizacion ZOH
Ad = np.array(sys_d.A); Bd = np.array(sys_d.B); Cd = np.array(sys_d.C)

# ---- OBSERVADOR (orden 2): polos dobles via acker sobre el aumentado
w_o = 40.0                                         # 1/h  ancho de banda␣
 ↪observador
z_o = np.exp(-w_o*T_s)                             # polo discreto equivalente
L   = np.array(acker(Ad.T, Cd.T, [z_o, z_o*0.999])).reshape(2, 1)

# ---- CONTROLADOR: lazo cerrado de 1er orden  y_dot = -w_c (y-r)
w_c = 5.0                                          # 1/h  ancho de banda control

print("b0   =", round(b0,4), "  T_s =", T_s, "h")
print("z_o  =", round(z_o,4), "  L =", L.flatten())
print("w_c  =", w_c, "1/h   (asentamiento ~", round(4/w_c,2), "h)")
print("eig(Ad - L*Cd) =", np.linalg.eigvals(Ad - L@Cd))

# ============================================================
# Perturbacion real BSM1  +  set-point programado
# ============================================================
Ss_func, Q1_func = make_disturbance_interp(BSM1_FILE)

def r_func(t_h):
    d = t_h/24.0
    if   d < 4:  return 2.0
    elif d < 8:  return 2.4
    elif d < 11: return 1.7
    else:        return 2.0

def rk4(y, t, h, u, Ss_f, Q1_f):
    k1 = dSO5_dt(t,     y,        u, Ss_f, Q1_f)
    k2 = dSO5_dt(t+h/2, y+h/2*k1, u, Ss_f, Q1_f)
    k3 = dSO5_dt(t+h/2, y+h/2*k2, u, Ss_f, Q1_f)
    k4 = dSO5_dt(t+h,   y+h*k3,   u, Ss_f, Q1_f)
    return y + (h/6)*(k1+2*k2+2*k3+k4)

# ============================================================
# Simulacion (14 dias del registro BSM1)
# ============================================================
T_END = 14*24.0; N = int(T_END/T_s); U_MIN, U_MAX = 0.0, 600.0
y  = 2.0
Qt0 = _Q1_nom + _Q2 + _Q3
f0 = ((1/V)*(_Q1_nom*S_O1 + _Q2*S_O2 + _Q3*S_O3) - (Qt0/V)*y + rZ(y, _Ss_nom))
z  = np.array([y, f0])              # [y_hat, f_hat] inicial

t_a=[]; y_a=[]; u_a=[]; r_a=[]; fT=[]; fH=[]
```

```
for k in range(N):
    t = k*T_s; r = r_func(t)
    y_h, f_h = z
    u0 = w_c*(r - y_h)                                  # control virtual (P sobre␣
 ↪salida estimada)
    u  = float(np.clip((u0 - f_h)/b0, U_MIN, U_MAX))# ley ADRC (+ anti-windup␣
 ↪por saturacion)
    fT.append(dSO5_dt(t, y, u, Ss_func, Q1_func) - b0*u)   # perturbacion total␣
 ↪real
    t_a.append(t); y_a.append(y); u_a.append(u); r_a.append(r); fH.append(f_h)
    y = rk4(y, t, T_s, u, Ss_func, Q1_func)          # avanza planta no lineal
    innov = y_a[-1] - (Cd @ z)[0]
    z = Ad @ z + Bd.flatten()*u + L.flatten()*innov # avanza observador (ESO)

t_a=np.array(t_a); y_a=np.array(y_a); u_a=np.array(u_a); r_a=np.array(r_a)
fT=np.array(fT); fH=np.array(fH); t_d=t_a/24.0
ISE = np.trapezoid((y_a-r_a)**2, t_a)
print("ISE = %.3f   RMS(y-r) = %.3f mg/L   DO final = %.3f (r=%.1f)"
      % (ISE, np.sqrt(np.mean((y_a-r_a)**2)), y_a[-1], r_a[-1]))

fig, ax = plt.subplots(2, 2, figsize=(13,8), layout='constrained')
fig.suptitle('ADRC digital (ESO orden 2) - regulacion de O2 disuelto con␣
 ↪perturbacion BSM1',
             fontsize=11, fontweight='bold')
ax[0,0].plot(t_d, r_a, 'b', lw=2, label='set-point r'); ax[0,0].plot(t_d, y_a,␣
 ↪'r--', lw=1, label='$S_O$')
ax[0,0].set_title('Oxigeno disuelto vs set-point'); ax[0,0].set_xlabel('dias');␣
 ↪ax[0,0].set_ylabel('mg/L'); ax[0,0].legend()
ax[0,1].plot(t_d, y_a-r_a, 'purple', lw=1); ax[0,1].axhline(0, c='k', lw=.7,␣
 ↪ls='--')
ax[0,1].set_title('Error de seguimiento  y - r'); ax[0,1].set_xlabel('dias');␣
 ↪ax[0,1].set_ylabel('mg/L')
ax[1,0].plot(t_d, u_a, 'darkorange', lw=.8, label='u'); ax[1,0].plot(t_d,␣
 ↪u_a*ALPHA, 'steelblue', lw=.8, label='$K_{La}$ [1/h]')
ax[1,0].set_title('Senal de control  u  y  $K_{La}=u/30$'); ax[1,0].
 ↪set_xlabel('dias'); ax[1,0].legend()
ax[1,1].plot(t_d, fT, 'k', lw=1, label='f real'); ax[1,1].plot(t_d, fH, 'r--',␣
 ↪lw=1.2, label=r'$\hat{f}$ (ESO)')
ax[1,1].set_title('Perturbacion total  f  y su estimacion'); ax[1,1].
 ↪set_xlabel('dias'); ax[1,1].set_ylabel('mg/(L h)'); ax[1,1].legend()
plt.show()
```

```
b0   = 0.2   T_s = 0.02 h
z_o  = 0.4493   L = [ 1.1017914  15.17430111]
w_c  = 5.0 1/h   (asentamiento ~ 0.8 h)
eig(Ad - L*Cd) = [0.44887964 0.44932896]
```

```
ISE = 1.065   RMS(y-r) = 0.056 mg/L   DO final = 2.041 (r=2.0)
```

